Impersonating and Reverting

09/15/2021

> **Note**
>
> This article applies to Windows.
>
> For information about ASP.NET Core, see ASP.NET Core Security.

Sometimes you might need to obtain a Windows account token to impersonate a Windows account. For example, your ASP.NET-based application might have to act on behalf of several users at different times. Your application might accept a token that represents an administrator from Internet Information Services (IIS), impersonate that user, perform an operation, and revert to the previous identity. Next, it might accept a token from IIS that represents a user with fewer rights, perform some operation, and revert again.

In situations where your application must impersonate a Windows account that has not been attached to the current thread by IIS, you must retrieve that account's token and use it to activate the account. You can do this by performing the following tasks:

1. Retrieve an account token for a particular user by making a call to the unmanaged **LogonUser** method. This method is not in the .NET base class library, but is located in the unmanaged **advapi32.dll**. Accessing methods in unmanaged code is an advanced operation and is beyond the scope of this discussion. For more information, see Interoperating with Unmanaged Code. For more information about the **LogonUser** method and **advapi32.dll**, see the Platform SDK documentation.

2. Create a new instance of the **WindowsIdentity** class, passing the token. The following code demonstrates this call, where `hToken` represents a Windows token.

   C#

   ```
   WindowsIdentity impersonatedIdentity = new WindowsIdentity(hToken);
   ```

3. Begin impersonation by creating a new instance of the WindowsImpersonationContext class and initializing it with the WindowsIdentity.Impersonate method of the initialized class, as shown in the following code.

   C#

```
WindowsImpersonationContext myImpersonation =
impersonatedIdentity.Impersonate();
```

4. When you no longer need to impersonate, call the WindowsImpersonationContext.Undo method to revert the impersonation, as shown in the following code.

```
C#
```

```
myImpersonation.Undo();
```

If trusted code has already attached a WindowsPrincipal object to the thread, you can call the instance method **Impersonate**, which does not take an account token. Note that this is only useful when the **WindowsPrincipal** object on the thread represents a user other than the one under which the process is currently executing. For example, you might encounter this situation using ASP.NET with Windows authentication turned on and impersonation turned off. In this case, the process is running under an account configured in Internet Information Services (IIS) while the current principal represents the Windows user that is accessing the page.

Note that neither **Impersonate** nor **Undo** changes the **Principal** object (IPrincipal) associated with the current call context. Rather, impersonation and reverting change the token associated with the current operating system process.

See also

- WindowsIdentity
- WindowsImpersonationContext
- Principal and Identity Objects
- Interoperating with Unmanaged Code
- ASP.NET Core Security